

DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

Implementación de TAD: Sistema de Fidelización AirMiles

November 6, 2026 — Algoritmos y Estructuras de Datos

Grupo 1

Integrante	LU	Correo electrónico
Garcia Ocio, Santiago	1210/25	santiagogarciaocio@gmail.com
Seda, Manuel	631/18	manu.seda@hotmail.com
García Peruchini, Federico	638/25	fede.gp2018@gmail.com
Pascazzi, Martin	711/25	martinpascazzi@gmail.com

Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires
Ciudad Universitaria - (Pabellón I/Planta Baja)
Intendente Güiraldes 2610 - C1428EGA
Ciudad Autónoma de Buenos Aires - Rep. Argentina
Tel/Fax: (++54 +11) 4576-3300
http://www.exactas.uba.ar

1 Diseño

El TAD AirMilles fue implementado con el siguiente diseño:

- MaxHeap de Usuarios.

Esta estructura es vital para cumplir las complejidades requeridas para la operacion RankingMillas. Como trade-off de agregar el Heap surgio el problema: Vincular a los Usuarios con la posicion en el Heap. Recordemos que el Heap es O(1) para obtener el maximo, pero para cualquier otro elemento es O(N).
Un primer intento de solucionar este problema consistió en agregar a cada Usuario un atributo con su posicion en el Heap, logrando asi ir a su posicion en O(1).
Sin emargo al momento de hacer shiftUp o shiftDown, operaciones necesarias para manter el invariante de la estructura cada vez que un usuario acunmula o canjea Millas, habria que modificar el indice de todos los usuarios afectados por el shift.
Por lo tanto necesitabamos poder acceder y modificar el indice de los Usuarios en O(1) para cualquier usuario. Esto nos llevo a la solucion adoptada finalmente. Guardar como atributo del heap dos arrays.
Uno de ellos, posicionesDeOrdenaHeap indica, dado un usuario (reconocido por su orden de ingreso al sistema) devuelve su indice actual en el Heap. Para obtener el orden de los Usuarios, el Heap tiene tambien un atributo orden. Un contador que va creciendo cuando ingresan nuevos usuarios. No podemos usar el atributo tamaño porque cuando realizamos la operacion de desencolar y encolar (para el metodo rankingMillas) los reingresos tendrían todos el mismo indice.
El otro posicionesDeHeapaOrden indica, dada una posicion del Heap el usuario que ubica esa posicion.

Esto nos permite dado un usuario, realizar las operaciones de shiftDown/shiftDown en, el peor caso en O(log N) si el usuario pasa del fin del Heap al Inicio. Y en cada intercambio de posiciones entre usuario y sus 'padres' las actualizaciones de sus indices se realizan en O(1).

Si no tuviesemos los indices los tendriamos que buscar en el ABB en tiempo O(Log N) para encontrarlo y O(log N) cantidad de busquedas por posibles padres. Finalmente con $O(\log(N)^2)$ excediendonos de complejidad O(log N) requerida en las operaciones.

- ABB que guarda la informacion de los Usuarios.
  - El ABB esta conectado con el Heap via Handle; y ordenado via el ID del Usuario.
- Trie de Promociones.

Implementado en forma de arreglo. En particular tenemos nodos de longitud 256. Cada posicion del nodo representa un caracter. Cuando se ingresa una neuva promocion cada caracter crea nuevos nodos iterativamente Al finalizar el nombre de la promocion el atributo 'Elem' pasa a contener dicha Promocion, que es una clase.

Para poder eliminar promociones, de forma tal que se eliminan todos los nodos que no llevan a ninguna promocion, utilizamos el atributo cantHijos. Este cuenta la cantidad de elementos que pasan por un determinado nivel del Trie. Una vez que dicho atributo nos marca 1, nos esta indicando que a partir de ese momento la unica promocion en ese camino del Trie es la que queremnos borrar.

- MasAcumulador.
  - El usuario que al momento es el que tiene mas millas totales.
- MasCanjeador.
  - El usuario que al momento gasto mas millas.

Promocion y Usuarios son Clases con sus correspondientes atributos y métodos.

2 Complejidades

Veamos que ese diseño nos permite cumplir con todas las complejidades requeridas para cada operacion.

- crearSistema en O(1)
  - Inicializar ABB O(1)
  - Inicializar Heap O(1)
  - Inicializar Trie O(1)
  - agregarPromocion('NoPromo') O(7) [Ver iniciarPromo]
  - Total = O(1)
- registrarUsuario / registrarUsuarioConReferidor en O(log U) caso promedio y O(U) peor caso
  - Encolamos Usuario en el Heap O(log U)
  - Agregamos el Handle del usuario al ABB; caso promedio O(log U), peor caso O(U)
  - Sumarle millas al usuario referidos es O(1) ya que lo que cuesta es encontrarlo
  - En el peor caso vamos vamos a tener que actualizar la posicion en el Heap del usuario referidor del piso al tope. Eso tiene costo de O(log U)
  - Costo total en promedio : O(log U) +O(log U) +O(1) + O(log U) = O(log U)
  - Costo total en peor : O(log U) +O(U) +O(1) + O(log U) = O(U)
- acumularMillas en O(log U + |L|) caso promedio y O(U + |L|) peor caso
  - Encontrar la promocion en el Trie: O(|L|)
  - Obtener la informacion de la promocion, restar del tope si es necesario O(1)
  - Eliminar promoción en caso de ser necesario O(|L|) [ver eliminarPromocion]
  - Encontrar al usuario en el ABB : caso promedio O(log U), peor caso O(U)
  - Aumentar los atributos correspondientes al usuario O(1)
  - Reajustar la posicion del Usuario en el Heap, de ser requerido, O(log U)
  - Costo total en promedio: O(|L|)+ O(1) +O(|L|)+ O(U) + O(1) + O(log U) = O(log U + |L|)
  - Costo total en peor: O(|L|)+ O(1) +O(|L|)+ O(log U) + O(1) + O(log U) = O(U + |L|)
- canjearMillas O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario en el ABB : caso promedio O(log U), peor caso O(U)
  - Realizar la operacion canjearMillas O(1)
  - Caso promedio : O(log U) + O(1) = O(log U),
  - Peor caso : O(U) +O(1) = O(U)
- transferirMillas en O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario transferidor en el ABB : caso promedio O(log U), peor caso O(U)
  - Encontrar al usuario receptor en el ABB : caso promedio O(log U), peor caso O(U)
  - Realizar el balance correspondiente del saldo de Millas de transferidor O(1)
  - Alterar el Heap para reflejar las nuevas millas del transferidor O(log U) en el peor caso

  - Actualizar, si fuera necesario, MasCanjeador O(1)
  - Realizar el balance correspondiente del saldo de Millas de receptor O(1)
  - Alterar el Heap para reflejar las nuevas millas del receptor O(log U) en el peor caso
  - Costo Promedio : O(log U) + O(log U) + O(1) + O(log U) + O(1) + O(1) + O(log U) = O(Log U)
  - Costo peor : O(U) + O(U) + O(1) + O(log U) + O(1) + O(1) + O(log U) = O(U)

- consultarSaldo / consultarCategoria en O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario en el ABB : caso promedio O(log U), peor caso O(U)
  - Consutlar su saldo/Categoria O(1)
  - Caso promedio : O(log U) + O(1) = O(log U),
  - Peor caso : O(U) +O(1) = O(U)

- iniciarPromocion / iniciarPromocionConTope en O(|L|)
  - Insertar elemento en Trie O(|L|)
  - En el peor caso por cada letra nueva hay que crear una nueva 'rama' del Trie. Eso tiene consto fijo dado por la longitud del arreglo O(256)
  - crear Promocion con o sin tope O(1)
  - costo total O(|L|) + O(1) = O(|L|)

- finalizarPromocion en O(|L|)
  - Buscar la promocion en el Trie O(|L|)
  - Eliminar la promocion si solamente se tiene que eliminar la promocion O(1), se se tiene que eliminar ramas O(|L|)
  - costo total pero caso O(|L|) + O(|L|) = O(|L|)

- elMasAcumulador / elMasCanjeador en O(1):
  - Pedirle informacion a MasAcumulador O(1)
  - Pedirle informacion a MasCanjeador O(1)

- rankingMillas en O(N log U)
  - Tomar N veces el maximo del Heap N*O(1)
  - Copiar los usuarios a un array O(N)
  - Volver a ingresar N veces los usuarios al Heap O(N log U)
  - Total: N*O(1) + O(N) + O(N log U) = O(N log U)

- paresTransferidoresExclusivos en O(U)
  - Recorrer el ABB elemento por elemento O(U)
  - Chequear para cada usuario si esta en parTransferidorExclusivo O(1)
  - Total : O(U)